

INSPECTAIR

Coding Standards & Rules

Luftqualitätsmessgerät mit ESP32-S3

Projekt: InspectAir – Luftqualitätsmessgerät
Version: 2.0
Datum: Februar 2026
Autoren: Maximilian Liebl, Cedric Stroh, Jannis Messer
Modul: Mikrocomputertechnik 1, DHBW Ravensburg
Dozent: Thomas Stingl (Airbus)

Dieses Dokument wurde unter Verwendung von KI-Tools (Claude, Anthropic) zur Überprüfung erstellt.

Inhaltsverzeichnis

1	Allgemeine Regeln	2
2	Namenskonventionen	2
3	Dokumentation (Doxygen)	2
4	Fehlerbehandlung	2
5	Git Workflow	3
6	Verbotene Praktiken	3
7	ESP32/FreeRTOS-spezifische Regeln	3
8	Const Correctness	3
9	Defensive Programmierung	4
10	Speicher-Management	4

1 Allgemeine Regeln

- **Sprache:** C++ (Arduino Framework via PlatformIO)
- **Einrückung:** 4 Spaces (keine Tabs)
- **Zeilenlänge:** max. 120 Zeichen
- **Encoding:** UTF-8
- Alle Dateien enden mit einer Leerzeile

2 Namenskonventionen

Element	Konvention	Beispiel
Variablen	snake_case	sensor_data
Konstanten	UPPER_SNAKE_CASE	CO2_THRESHOLD
Funktionen	snake_case	sensors_init()
Structs/Enums	PascalCase	SensorData
Dateien	snake_case.cpp/.h	sensors.cpp
Defines	UPPER_SNAKE_CASE	#define PIN_SDA 8

3 Dokumentation (Doxygen)

Alle öffentlichen Funktionen und Structs müssen dokumentiert werden:

```
/**
 * @brief Initialisiert alle Sensoren
 *
 * Initialisiert I2C, UART und alle angeschlossenen Sensoren.
 * Bei Fehler wird das entsprechende ok-Flag auf false gesetzt.
 *
 * @param config Zeiger auf Konfigurationsstruktur
 * @return true wenn mindestens ein Sensor verfuegbar
 * @return false wenn kein Sensor initialisiert werden konnte
 */
bool sensors_init(const SensorConfig* config);
```

4 Fehlerbehandlung

- Jeder Sensor hat ein eigenes ok-Flag im **SensorData** Struct
- Sensor-Fehler werden geloggt aber nicht als fatal behandelt
- Display zeigt -- für nicht verfügbare Werte
- Kritische Fehler (Display-Init) führen zu Halt

5 Git Workflow

- **Main-Branch:** Nur getesteter, funktionierender Code
- **Feature-Branches:** `feature/sensor-module`, `feature/ui-detailed`

Commit-Messages mit Präfix:

Präfix	Bedeutung
<code>feat:</code>	Neue Funktionalität
<code>fix:</code>	Bugfix
<code>docs:</code>	Dokumentation
<code>refactor:</code>	Code-Umstrukturierung

6 Verbotene Praktiken

- X** Keine Magic Numbers – immer `#define` oder `const` verwenden
- X** Keine globalen Variablen ohne `static` (Datei-Scope begrenzen)
- X** Keine `delay()` in `loop()` – `millis()` oder `yield()` verwenden
- X** Keine `String`-Klasse – `char[]` Buffer verwenden
- X** Keine dynamische Speicherallokation in `loop()`
- X** Kein `Serial.print()` in ISRs

7 ESP32/FreeRTOS-spezifische Regeln

- `volatile` für alle Variablen die in ISRs gelesen/geschrieben werden
- `IRAM_ATTR` für Interrupt Service Routines
- Task-Stack-Größe beachten (Standard: 4KB, bei viel lokalem Speicher erhöhen)
- `portENTER_CRITICAL()` / `portEXIT_CRITICAL()` für shared resources zwischen Tasks

8 Const Correctness

- Pointer-Parameter die nicht geändert werden: `const char* text`
- Methoden die Objekt nicht ändern: `int getValue() const`
- Lokale Variablen die nicht geändert werden: `const int max_retries = 10`

9 Defensive Programmierung

- Null-Pointer-Checks vor Dereferenzierung
- Array-Bounds validieren
- Timeout bei blockierenden Operationen (WiFi, Sensor-Reads)
- Sinnvolle Default-Werte bei fehlgeschlagenen Reads

10 Speicher-Management

- Statische Allokation bevorzugen
- Dynamische Allokation nur in `setup()` / `begin()` Funktionen
- Buffer-Größen als Konstanten definieren
- Bei Arrays: `sizeof()` statt hardcoded Größen